

Filtros, Busca e Paginação

Consultando dados com mais precisão: evolua o endpoint de listagem da API com filtros, busca textual e paginação para construir consultas controladas, eficientes e adequadas para aplicações reais.

BACKEND

REST API

NESTJS

Missão: Por Que Evoluir a Listagem?

Até aqui, o endpoint de listagem retornava **todos os registros cadastrados**. Esse comportamento funciona em uma base pequena, mas não representa o uso de uma aplicação real.

À medida que o número de registros cresce, a API precisa oferecer formas de consultar dados com mais precisão. O objetivo deste material é implementar e testar **filtros, busca e paginação** no endpoint de listagem.

Listar dados não significa apenas retornar tudo do banco, mas construir uma consulta controlada, eficiente e adequada para diferentes necessidades.

O que você vai aprender

- Utilizar query parameters em endpoints REST
- Implementar busca textual simples
- Aplicar filtros opcionais
- Adicionar paginação à listagem
- Retornar metadados úteis na resposta
- Testar combinações de consulta com REST Client

Onde Estamos no CRUD

Antes de evoluir, veja o ciclo básico já consolidado e o próximo passo que vamos dar:



Create

Read all

Read by ID

Update

Delete

O endpoint de listagem atual provavelmente está assim:

Antes

```
GET /products
```

Retorna **todos** os registros sem critério.

Depois

```
GET /products?search=arroz&page=1&limit=10
```

Retorna registros **filtrados, buscados e paginados**.

O Problema de Listar Tudo

Listar todos os registros pode parecer simples, mas traz problemas sérios quando a base cresce.

50

Registros

Ainda gerenciável, mas já sem controle

500

Registros

Respostas começam a ficar pesadas

5K

Registros

Lentidão perceptível na API

50K

Registros

Inaceitável sem paginação

Respostas Gigantes

Payloads muito grandes trafegam pela rede desnecessariamente.

Lentidão na API

O servidor processa e serializa mais dados do que o necessário.

Alto Consumo de Memória

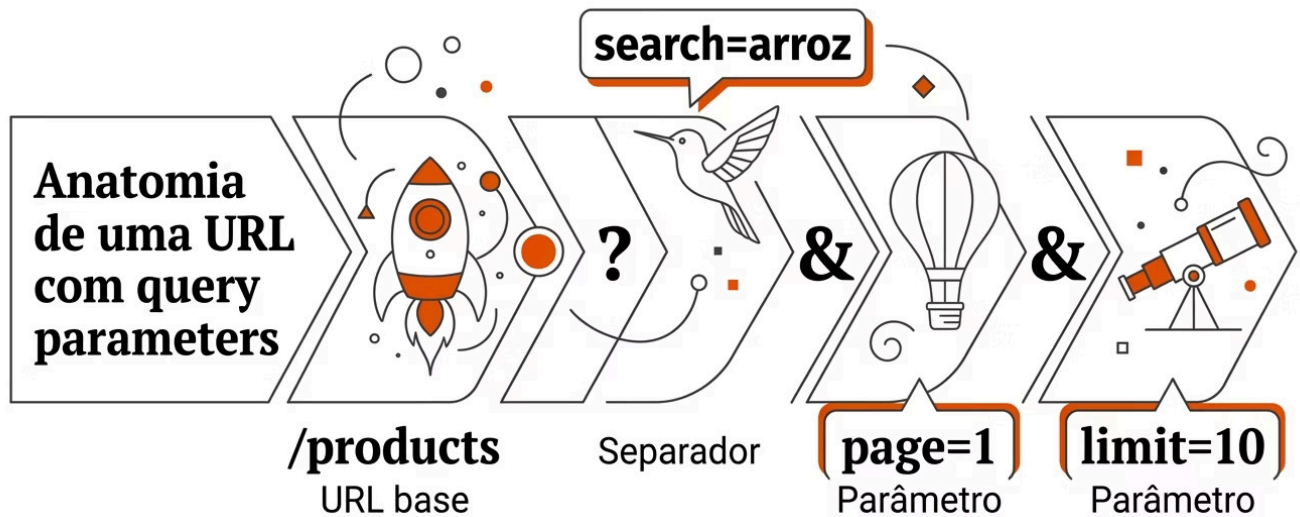
Todos os registros ficam carregados em memória ao mesmo tempo.

Experiência Ruim

Quem consome a API recebe mais dados do que precisa.

O Que São Query Parameters

Query parameters são informações enviadas na URL depois do sinal `?`. São a principal forma de passar critérios de consulta em endpoints REST.



Um parâmetro

```
GET /products?search=arroz
```

`search` é o nome do parâmetro e `arroz` é o valor.

Múltiplos parâmetros

```
GET /products?search=arroz&page=1&limit=10
```

Usamos `&` para separar cada parâmetro adicional.

Os três parâmetros do exemplo: `search = arroz` | `page = 1` | `limit = 10`

Busca Textual e Filtros Opcionais

Busca Textual

Permite consultar registros a partir de uma palavra ou trecho de texto. A busca não exige correspondência exata — basta encontrar o termo dentro do campo.

```
GET /products?search=arroz
```

A API procura o termo nos campos:

- `name` contém "arroz"
- `description` contém "arroz"

Em ORMs, isso pode ser feito com operadores como `Like` ou `ILike`:

```
where: [  
  { name: Like(`%${query.search}%`) },  
  { description: Like(`%${query.search}%`) },  
]
```

Filtros Opcionais

Filtros restringem os resultados por critérios específicos. O ponto principal é que **devem ser opcionais** — se não informados, a API não filtra por aquele campo.

```
GET /products?active=true  
GET /products?categoryId=123  
GET /products?minPrice=10&maxPrice=50
```

Exemplo de aplicação no service:

```
const where: any = {};  
  
if (query.active !== undefined) {  
  where.active =  
    query.active === true ||  
    query.active === 'true';  
}
```

Comece com poucos filtros, de acordo com os campos reais da entidade do projeto.

Paginação: Dividindo os Resultados

Paginação é a técnica de dividir os resultados em páginas menores. Em vez de retornar todos os registros, a API retorna apenas uma parte controlada.

```
GET /products?page=1&limit=10
```

page=1 & limit=10

Retorna registros 1 a 10

page=3 & limit=10

Retorna registros 21 a 30

1

2

3

4

page=2 & limit=10

Retorna registros 11 a 20

page=N & limit=10

Retorna os próximos 10 registros

O cálculo do `skip` (quantos registros pular) é feito com a fórmula:

$$skip = (page - 1) \times limit$$

Estrutura da Resposta Paginada

Quando usamos paginação, é útil retornar os dados e informações adicionais sobre a consulta. Isso ajuda quem consome a API a saber se há mais páginas disponíveis.

Exemplo de resposta

```
{
  "data": [
    {
      "id": "1",
      "name": "Produto A"
    },
    {
      "id": "2",
      "name": "Produto B"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,

```

```
"total": 25,  
"totalPages": 3  
}  
}
```

O que cada campo significa

data

Array com os registros retornados nesta página.

meta.page

Número da página atual solicitada.

meta.limit

Quantidade de registros por página.

meta.total

Total de registros que correspondem à consulta.

meta.totalPages

Total de páginas disponíveis: `Math.ceil(total / limit)`

Implementação: DTO, Controller e Service

Criar o DTO de Consulta

Assim como usamos DTOs para criação e atualização, usamos um DTO para os parâmetros de consulta. Isso evita que a lógica fique espalhada e sem documentação clara.

```
export class FindProductsQueryDto {  
  search?: string;  
  page?: number;  
  limit?: number;  
  active?: boolean;  
}
```

Receber no Controller com @Query()

O controller continua simples: recebe os parâmetros e delega a regra de consulta ao service.

```
@Get()  
findAll(@Query() query: FindProductsQueryDto) {  
  return this.productsService.findAll(query);  
}
```

Implementar a Consulta no Service

O service interpreta os parâmetros recebidos e monta a consulta com paginação, filtros e busca.

```
async findAll(query: FindProductsQueryDto) {
  const page = Number(query.page) || 1;
  const limit = Math.min(Number(query.limit) || 10, 50);
  const skip = (page - 1) * limit;

  const [data, total] = await this.productsRepository.findAndCount({
    where,
    skip,
    take: limit,
  });

  return {
    data,
    meta: { page, limit, total,
    totalPages: Math.ceil(total / limit) },
  };
}
```

Valores Padrão e Limite Máximo

Valores Padrão

A API deve definir valores padrão para paginação. Assim, se o usuário chamar `GET /products` sem parâmetros, a API ainda retorna uma resposta paginada controlada.

page padrão

1 — sempre começa da primeira página

limit padrão

10 — retorna 10 registros por página

Isso evita que a ausência de parâmetros gere uma resposta descontrolada.

Limite Máximo por Página

Também é importante definir um limite máximo para evitar que alguém solicite um número absurdo de registros.

`GET /products?page=1&limit=10000`

A API deve reduzir automaticamente para o máximo permitido:

```
const limit = Math.min(  
  Number(query.limit) || 10, 50 // limite máximo  
);
```

Se o `meta.limit` retornado for diferente do solicitado, o consumidor da API saberá que o limite foi ajustado.

Testes no REST Client: Visão Geral

O arquivo `.http` deve ser organizado com uma seção dedicada para os testes de filtros, busca e paginação. Veja a estrutura sugerida:

QUERY 1

Listagem paginada padrão

```
GET {{baseUrl}}/products
```

QUERY 2

Primeira página com limite 5

```
GET {{baseUrl}}/products?page=1&limit=5
```

QUERY 3

Segunda página com limite 5

```
GET {{baseUrl}}/products?page=2&limit=5
```

QUERY 4

Busca por texto

```
GET {{baseUrl}}/products?search=arroz
```


QUERY 6-8

Combinação, limite alto e página vazia

```
GET {{baseUrl}}/products?search=arroz&active=true&page=1&limit=5
```

Testes 1 a 4: Paginação e Busca

Listagem Paginada Padrão

```
GET {{baseUrl}}/products
```

- A resposta possui data e meta?
- A página padrão é 1 e o limite padrão foi aplicado?
- O total de registros e de páginas aparece?

Primeira Página com Limite 5

```
GET {{baseUrl}}/products?page=1&limit=5
```

- Foram retornados no máximo 5 registros?
- meta.page é 1 e meta.limit é 5?

Segunda Página com Limite 5

```
GET {{baseUrl}}/products?page=2&limit=5
```

- Os registros são diferentes dos da primeira página?
- O total permaneceu o mesmo?

Busca Textual

```
GET {{baseUrl}}/products?search=arroz
```

- Apenas registros relacionados ao termo foram retornados?
- A busca considera name e description?
- A busca funciona mesmo com paginação?

Testes 5 a 8: Filtros e Casos Extremos

Teste 5 — Filtro Simples

```
GET {{baseUrl}}/products?active=true
```

- A API retornou apenas registros ativos?
- O filtro foi ignorado quando não informado?
- O valor `false` também funciona?

Teste 6 — Combinação Completa

```
GET {{baseUrl}}/products?search=arroz&active=true&page=1&limit=5
```

- Busca, filtro e paginação foram aplicados?
- A resposta continua organizada em `data` e `meta`?

Teste 7 — Limite Muito Alto

```
GET {{baseUrl}}/products?page=1&limit=10000
```

- A API aplicou um limite máximo?
- A resposta continua estável?
- `meta.limit` mostra o limite realmente usado?

Teste 8 — Página Sem Resultados

```
GET {{baseUrl}}/products?page=999&limit=10
```

Comportamento esperado:

```
{
  "data": [],
  "meta": {
    "page": 999,
    "limit": 10,
    "total": 25,
    "totalPages": 3
  }
}
```

Retornar lista vazia é aceitável, desde que os metadados estejam corretos.

Cuidados Importantes na Implementação



A API deve continuar **previsível e fácil de testar**. Nunca misture regra de consulta diretamente no controller — mantenha essa lógica no service.

Critérios de Sucesso e Checklist

✓ Critérios de Sucesso

Este material pode ser considerado concluído quando:

- ✓ GET /products aceita page e limit
- ✓ A API define valores padrão para paginação
- ✓ A API limita a quantidade máxima por página
- ✓ A resposta retorna data e meta
- ✓ O total de registros é informado
- ✓ O total de páginas é calculado
- ✓ A busca textual funciona quando search é informado
- ✓ Os filtros opcionais funcionam quando informados
- ✓ A ausência de filtros não quebra a listagem
- ✓ O REST Client possui testes para os principais cenários



Checklist de Implementação

Antes de finalizar, verifique se:

- ☐ Foi criado um DTO para query parameters
- ☐ O controller recebe os parâmetros com `@Query()`
- ☐ O service concentra a lógica de consulta
- ☐ A paginação usa `page`, `limit` e `skip` corretamente
- ☐ O `limit` possui valor padrão
- ☐ O `limit` possui limite máximo
- ☐ A busca textual foi testada
- ☐ Os filtros opcionais foram testados
- ☐ A resposta possui data e meta
- ☐ O arquivo `.http` foi atualizado com os novos testes

Checkpoint de Entendimento

Ao final deste material, você deve conseguir responder às seguintes perguntas:

Por que listar todos os registros pode ser um problema?

Porque à medida que a base cresce, respostas muito grandes causam lentidão, alto consumo de memória e tráfego de rede desnecessário.

O que são query parameters?

São informações enviadas na URL após o sinal `?`, usadas para passar critérios de consulta como `search`, `page` e `limit`.

Qual é a diferença entre busca textual e filtro?

A busca textual procura um termo dentro de campos de texto. O filtro restringe resultados por um valor exato de um campo específico, como `active=true`.

Como calcular quantos registros devem ser ignorados?

Com a fórmula `skip = (page - 1) * limit`. Na página 2 com limite 10, pulamos os primeiros 10 registros.

Por que uma resposta paginada deve trazer metadados?

Para que quem consome a API saiba o total de registros, quantas páginas existem e se há mais dados disponíveis além da página atual.

Por que definir um limite máximo por página?

Para evitar que requisições com `limit=10000` sobrecarreguem o servidor, garantindo estabilidade e previsibilidade da API.

Resumo Final e Próximos Passos

Neste material, evoluímos a operação de listagem da API com filtros, busca e paginação. O endpoint de listagem deixou de ser apenas:

```
GET /products
```

E passou a aceitar consultas mais completas e controladas:

```
GET /products?search=arroz&active=true&page=1&limit=5
```

Query Parameters

Aprendemos a receber e interpretar parâmetros de consulta na URL com @Query() e DTOs dedicados.

Resposta Paginada

A resposta agora retorna data com os registros e meta com informações de paginação.

Valores Padrão e Máximos

Definimos page=1 e limit=10 como padrão, e limit=50 como máximo permitido.

Testes no REST Client

Oito cenários de teste cobrem paginação, busca, filtros, limites extremos e páginas vazias.

Próximo passo: Aprofundar as validações avançadas e o tratamento de erros sem autenticação, melhorando a robustez da API diante de entradas inválidas e cenários inesperados.